

Department of Mathematics and Computer Science
Eindhoven University of Technology

CBL Java Application Development

Course: Programming (2IP90)

Academic Year 2026–2027

Prepared by Iman Hemati Moghadam



1. Challenge Description

In this Challenge-Based Learning (CBL) project, you will work in **pairs** to design and implement an interactive Java application with a **Graphical User Interface (GUI)** during Weeks 5–8 of the course. **You are free to choose the type of application you want to develop, including a game or another type of interactive application.**

The project is designed to help you apply the core programming concepts covered in the course within a larger, open-ended software design context. You are expected to design, implement, test, and continuously improve your application while collaborating effectively with your partner.

At the start of the project, each pair will be assigned a dedicated tutor. During your first meeting, you will present and discuss your project idea with your tutor to ensure that its scope and technical complexity are appropriate for the available timeframe and aligned with the course learning goals. You will also have regular meetings with your tutor during the practical sessions to discuss your progress, receive feedback, and address technical or design-related questions.

Learning Goals

This project provides an opportunity to develop and demonstrate the following skills and competencies:

- **Object-Oriented Design:** Design and implement a Java application using sound object-oriented programming (OOP) principles, including appropriate classes, objects, and responsibilities.
- **Modular Architecture:** Apply modular software architecture and structured development practices to organise a larger-scale project into clear and manageable components.
- **GUI Development:** Design and develop an interactive and functional graphical user interface using Java Swing.
- **Problem Solving:** Integrate multiple software components and independently analyse and solve technical challenges that arise during development.
- **Independent Research:** Independently research, learn, and apply two advanced technical topics that extend beyond the standard course material.
- **Code Quality:** Produce readable, well-structured, maintainable, and appropriately documented code following good programming practices.
- **Collaboration:** Collaborate effectively with your partner and communicate your progress, decisions, challenges, and questions clearly with your tutor.

2. Group Formation & Tutor Selection

2.1 Partner Selection & Team Changes

As this is a pair-based project, you will work in **teams of two**. We strongly recommend continuing to work with your partner from the first four weeks of the course, as you already have an established workflow and experience working together.

If you prefer to form a new partnership, email **2ip90@tue.nl** and CC your current teammate, your proposed new teammate, and their current partner. This ensures that **all students affected by the change are informed of the proposed new team composition**. In your email, clearly state that you would like to form a new team and identify the students who will make up the new team.

Once the email has been received, we will register you in your new team. Please note that changing partners may require you to move to a different practical session with another instructor/TA.

Team Change Deadline

Strict Deadline: Thursday, September 24, 2026 at 23:55.

Requests submitted after this deadline will not be processed to maintain schedule stability.

2.2 Tutor Selection & Supervision

As mentioned earlier, one member of the teaching team (instructor/TA) will support your group throughout the project. Your tutor will supervise your group, provide iterative feedback, help you keep your project on track, and answer technical or project-related questions. Tutors may also ask you to explain your implementation and design decisions during project meetings to ensure that you understand the work you have submitted. **Your tutor will also grade your final submission.**

Tutor registration will take place during the practical sessions in Week 4, on **Tuesday, September 22, 2026, or Thursday, September 24, 2026**. You must register **during one of these practical sessions** by informing the room instructor of your preferred tutor.

Tutor registration by email is not accepted. Registrations must be made in person during the specified practical sessions.

You may select your tutor from **the instructor and two TAs assigned to your practical session**. Tutors have limited capacity, so if your preferred tutor is already at capacity, you will need to select another tutor.

Groups that do not register for a tutor will be **randomly assigned to a tutor based on the remaining available capacity**. **Please note that this may require your group to move to a different practical session and room**. If you prefer to work with the **instructor or TAs assigned to your current room** and remain in your current practical session, **make sure to register during one of the designated sessions**.

3. Application Design & Implementation

You will design and implement an **interactive Java application using a Java Swing GUI**. You may develop your own application idea or **take inspiration from an existing application**. You are not expected to implement a complete or fully featured application. Instead, you should select and implement a suitable set of features that provides sufficient scope for this project.

Your application should include:

1. **Clear purpose:** The application should have a clear purpose or objective that users can understand.
2. **User interaction:** Users should be able to interact meaningfully with the application through the GUI.
3. **Core functionality:** The application should provide meaningful functionality appropriate to its purpose.
4. **Application state:** The application should maintain and update relevant state as users interact with it.
5. **Feedback:** The application should provide clear feedback in response to user actions.
6. **Completion or outcome:** The application should have a meaningful outcome, result, or completion condition appropriate to its purpose.

When planning your application, consider which features and aspects you want to implement and how they contribute to the overall user experience. Discuss your proposed application and selected features with your tutor. Your tutor will help you **refine the scope and complexity** so that it is appropriate for your programming knowledge, the project learning goals, and the available **four-week development period**.

The final project should implement enough meaningful functionality to demonstrate the programming concepts and skills described in the learning goals.

4. Advanced Topics & Independent Research

As part of this project, you are required to go beyond the standard course material by independently researching and implementing **two advanced topics of your choice**. The selected topics should either extend concepts introduced in the course to a more advanced level or introduce entirely new concepts that are relevant to your project.

Early Approval: Discuss your chosen topics with your tutor early in the project to confirm that their technical depth, relevance, and scope are appropriate for the project and the available timeframe.

As part of the final submission, you must submit a **written reflection of a maximum of two pages in PDF format** covering both selected topics.

For each topic, briefly describe:

- **Why** you chose the topic and how it is relevant to your project.
- **What** you learned through your research, including the main concepts or principles.
- **How** you applied the topic in your project.
- **What you learned from the implementation**, including challenges, decisions, and the impact on your project.

The reflection should demonstrate your **understanding of the topics and their meaningful application** in your project, rather than simply describing what you implemented.

Example Topics

The following are examples of suitable topics. This list is **not exhaustive**; you may propose other topics, provided that they are sufficiently advanced and relevant to your project.

- **Software Engineering:** Test-Driven Development (TDD), Version Control Workflows (Git / Branching Strategies), Build Systems (Gradle / Maven), Continuous Integration (CI).
- **Design & Architecture:** Advanced Object-Oriented Design Patterns, User Experience (UX) & Game Design, Advanced Data Structures & Algorithms.
- **Language Features:** Modern Java Features (Streams, Functional Interfaces & Lambda Expressions), Concurrency & Multithreading.
- **Testing & Quality:** Unit Testing (JUnit), Integration Testing, Mocking, Static Code Analysis, Code Coverage.

5. Deliverables & Project Updates

5.1 Product Backlog

Before starting detailed development, you must create a Product Backlog. The backlog consists of prioritised features (backlog items) structured into manageable, independent development tasks. Each backlog item must contain:

- **Name:** A succinct, descriptive title (e.g., “Rendering a chessboard”).
- **How to Demo:** Exact step-by-step instructions for verifying the functionality (e.g., “Launch the app, click New Game; an 8×8 checkerboard is displayed and resizes correctly when the window size changes”).
- **Notes:** Relevant technical notes, edge cases, or implementation details.

Discussion with Your Tutor: During the practical session on **Tuesday, September 29**, you can discuss your project idea and proposed backlog with your tutor. Your tutor can provide guidance on how to structure the backlog and help you determine whether the proposed scope is appropriate for the project. You can then refine your Product Backlog based on this discussion and **submit it in Canvas by Tuesday, September 29, 2026 at 23:59**. Examples of Product Backlogs are provided in **Section 9** at the end of this document.

The Product Backlog is not graded. Its purpose is to establish a shared understanding between your group and your tutor about the planned project before you proceed with detailed implementation. The backlog is a **living document** and should be refined and updated as the project progresses, and the final version must be included in the final project submission.

5.2 Project Presentations

Groups will give three short **oral presentations of up to 5 minutes** during scheduled practical sessions. The purpose of these presentations is to communicate your project plans, report progress, and receive feedback from your tutor and peers.

Slides are not required. You may use your printed Product Backlog as a reference during the presentations.

- **Presentation 1 (Thursday, October 1, 2026):**

Present your project idea, core mechanics, and planned features in more detail. Explain the features listed in your Product Backlog and discuss how you plan to implement them. This presentation builds on the project discussion with your tutor on **Tuesday, September 29**, where you discussed your initial idea and project scope.

- **Presentation 2 (Thursday, October 8, 2026):**

Provide a progress update, including completed features, current challenges or technical issues, and your implementation goals for the next phase of the project.

- **Presentation 3 (Thursday, October 15, 2026):**

Provide a progress update, including completed features, current challenges or technical roadblocks, and your implementation goals for the final phase of the project.

Attendance is mandatory. Missing a presentation will result in a deduction of **5 points** from the 25-point CBL project grade. Therefore, missing all three presentations would result in a deduction of **15 points**.

Regular attendance is also important for the individual grading process. These presentations allow your tutor to become familiar with your group, your project, and your individual contributions. If you do not attend, your tutor has fewer opportunities to assess your understanding of the project and your implementation work.

5.3 Final Submission Requirements

Final Hand-in Guidelines

Submission Deadline: **Friday, October 23, 2026 at 23:59 via Canvas.**

Deliverable: Submit **one ZIP archive** containing the following:

1. **Runnable Java project and full source code.**
2. **README.md** containing clear instructions for building and running the application.
3. Final version of the **Product Backlog**.
4. **Written reflection document** on the two independently researched topics of choice.

Execution Criteria: The application must build and run on **Windows, macOS, and Linux** using Java Swing. The application must be **fully usable within 1 minute of launching**, without requiring additional configuration beyond the instructions provided in the README.

6. Assignment Timeline

The project is carried out over Weeks 5–8. Students are expected to **work on the project throughout this period**. **Table 1** provides an overview of all key project milestones and deadlines.

Date & Event Location	Milestone / Task
Tuesday, September 29, 2026 Practical Session	Product Backlog discussion with tutor & initial project scope
Tuesday, September 29, 2026 at 23:59 Canvas	Product Backlog submission deadline
Thursday, October 1, 2026 Practical Session (Mandatory)	Presentation 1: Project idea, core mechanics, and planned features
Tuesday, October 6, 2026 Practical Session	Stand-up Meeting & Guided Practical Development - Ask Your Tutor Questions and Get Guidance
Thursday, October 8, 2026 Practical Session (Mandatory)	Presentation 2: Progress update, current challenges, and implementation goals
Tuesday, October 13, 2026 Practical Session	Stand-up Meeting & Guided Practical Development - Ask Your Tutor Questions and Get Guidance
Thursday, October 15, 2026 Practical Session (Mandatory)	Presentation 3: Progress update, current challenges, and goals for the final implementation phase
Tuesday, October 20, 2026 Practical Session	Stand-up Meeting & Guided Practical Development - Ask Your Tutor Questions and Get Guidance. Important: Some rooms have changed. Check your assigned room for this session in the Room Schedule
Thursday, October 22, 2026 Practical Session	Final Testing, Refinement & Stand-up Meeting Important: Some rooms have changed. Check your assigned room for this session in the Room Schedule .
Friday, October 23, 2026 at 23:59 Canvas	Final Project Submission Deadline

Table 1: Project Timeline & Milestone Schedule

Presentation Attendance:

Attendance at all three presentations is mandatory. See **Section 5.2 Project Presentations** for the attendance requirements and deduction policy.

Room Changes: On **20 and 22 October**, some practical sessions will take place in different rooms. Please check the [Room Schedule](#) to make sure you attend the correct room.

7. Assessment Rubric

Criterion	Excellent	Good	Sufficient	Insufficient
OOP & Modularity	6 pts Clear architectural design with strong use of OOP principles, appropriate abstraction, and well-separated components.	4–5 pts Good overall class structure and separation of responsibilities, with some minor design issues.	2–3 pts Some modular structure, but significant coupling or limited use of OOP principles.	0–1 pt Poorly organized, or inappropriate object-oriented design.
Functionality	6 pts All planned core features are implemented and work correctly. The application is stable and behaves reliably during normal use, with no significant functional or runtime issues.	4–5 pts Most planned features are implemented and work correctly. Minor functional or runtime issues may occur but do not significantly affect normal use.	2–3 pts Basic functionality is implemented, but several features are incomplete or significant bugs affect normal use.	0–1 pt Limited functionality is implemented, with major bugs or missing core features.
GUI Implementation	4 pts Well-designed, intuitive, responsive, and fully functional Java Swing GUI appropriate for the application.	3 pts Functional Java Swing GUI with appropriate layout and basic design considerations.	1–2 pts Limited, inconsistent, or partially non-functional GUI components.	0 pts GUI is missing, largely non-functional, or implemented using an inappropriate framework.
Independent Topics	6 pts Both topics demonstrate substantial independent research, strong understanding, and effective integration into the project. Students can clearly explain how and why the topics were applied.	4–5 pts Both topics are correctly implemented with a reasonable level of research, understanding, and integration.	2–3 pts Topics are only superficially explored or implemented, with limited understanding or integration.	0–1 pt Required topics are missing, incorrectly implemented, or demonstrate insufficient understanding.
Code Quality	3 pts Clean, readable, consistently formatted, and maintainable code with appropriate naming and documentation.	2 pts Mostly readable and maintainable code with minor inconsistencies in formatting, naming, or documentation.	1 pt Inconsistent naming, formatting, structure, or limited documentation reduces readability and maintainability.	0 pts Code is difficult to read, poorly organized, or inadequately documented.

Total Assessment Score: 25 Points

9. Example Product Backlogs

The following examples illustrate how a Product Backlog can be structured. They are examples only; you should create your own backlog based on your chosen application.

Each backlog item describes a specific piece of functionality that can be implemented and demonstrated independently. The backlog should be detailed enough to guide development, but it can be refined as the project progresses.

Example Product Backlog: Connect Four

Possible advanced topics: JUnit testing, Git branching workflows.

#	Name	How to Demo	Notes
1	Display game board	Launch the app; the game board is displayed clearly in the GUI.	Display a 7×6 board with clearly distinguishable cells.
2	Start a new game	Launch the app and click New Game; an empty board is displayed and the game is ready.	Reset the board and initialise the game state.
3	Place a piece	Start a game and select a valid column; the current player's piece appears in the lowest available cell.	A piece cannot be placed in a full column.
4	Switch players	Place a valid piece; the active player changes and the GUI indicates whose turn it is.	Maintain the current-player state.
5	Detect winning move	Place pieces that create four in a row; the application detects the win and reports the result.	Check horizontal, vertical, and diagonal connections.
6	Prevent invalid moves	Fill a column and try to place another piece in it; the application prevents the move and gives feedback.	Handle full columns and other invalid actions.
7	End the game	Win a game or fill the board without a winner; the application displays the appropriate result and stops further moves.	Handle both win and draw conditions.

Example Product Backlog: Interactive Quiz

Possible advanced topics: Java Streams and Enums and Java Records.

#	Name	How to Demo	Notes
1	Display a question	Launch the app; a question and its available answer options are displayed.	Store questions and possible answers in a suitable data structure.
2	Select an answer	Display a question and select one of the available answers; the selected answer is clearly indicated.	Only one answer can be selected at a time.
3	Submit an answer	Select an answer and click Submit; the application checks the answer and provides feedback.	Indicate whether the selected answer is correct.
4	Move to next question	Submit an answer and click Next; the next question is displayed.	Do not skip questions or repeat them unintentionally.
5	Track score	Answer several questions; the score is updated according to the number of correct answers.	Maintain the score as application state.
6	Display final result	Complete the quiz; the application displays the final score and a meaningful result.	Show the number of correct answers and/or a percentage.
7	Restart quiz	Complete a quiz and click Restart; the questions and score are reset, and a new quiz begins.	Reset all relevant application state.

Example Product Backlog: Personal Expense Tracker

Possible advanced topics: file handling and Object-Oriented Design Patterns.

#	Name	How to Demo	Notes
1	Display expense list	Launch the app; the current expenses are displayed in a clear list or table.	Show relevant information such as description, amount, and category.
2	Add an expense	Enter an expense and click Add; the new expense appears in the list.	Validate required input such as description and amount.
3	Remove an expense	Select an expense and click Remove; the selected expense is removed from the list.	Prevent removal when no expense is selected.
4	Calculate total expenses	Add several expenses; the application displays the correct total amount.	Update the total whenever the expense list changes.
5	Categorise expenses	Add an expense and select a category; the expense is displayed with the selected category.	Use a fixed set of suitable categories.
6	Filter expenses by category	Add expenses from different categories and select a category filter; only matching expenses are displayed.	Allow the user to return to the full list.
7	Clear all expenses	Add several expenses and click Clear All; the expense list and total are reset.	Ask for confirmation before deleting all expenses.

Using These Examples

A backlog does not need to contain exactly seven items. The number of items should reflect the chosen application and the available four-week development period. Each item should describe a meaningful, manageable feature that can be developed and demonstrated independently where practical.